מקרא:
טקסט המסומן בצבע ירוק – הרחבות למי שמעוניין בכך.
החומר המתואר בחלקים אלו אינו מהווה חלק מהחומר המחייב לבחינה.

Files Ownership: בעלות על קובץ כבסיס לאבטחה

בעלות (Ownership) – העיקרון המכונן במודל האבטחה של לינוקס.

מודל Multi-User:
- לינוקס תוכננה לכתחילה לשרת משתמשים רבים בו-זמנית.
- היינו, לאפשר למשתמשים שונים להריץ בו זמנית תכניות [= תהליכים] באותו מחשב !
- לפיכך, לכל תהליך יש **בעלים** [= **owner**]
- כנ"ל גם לכל קובץ (רגיל, ספרייה וכו') במערכת הקבצים יש בעלים מוגדר אחד [= **File Owner**] והוא מהווה את הבסיס למודל האבטחה במערכת הקבצים של לינוקס.

- הבעלות מאפשרת למשתמש A להגן על קבציו מפני שינוי או קריאה על ידי משתמש B, מבלי להסתמך על תהליכים.

1. זיהוי ורישום הבעלות: UID ו- GID

זיהוי משתמש:
- כל משתמש במערכת מיוצג על ידי User Name (מחרוזת) ו- User ID (UID) (מספר שלם)
- היינו, לכל משתמש במערכת יש 'זהות כפולה'.

- כמו כן, לכל קובץ יש בעלים [= owner]
- ה- UID של הבעלים של הקובץ שמור ברשומת Inode של הקובץ [= ה- Inode שבו שמורים נתוני ה- metadata של הקובץ]:
- כאשר קובץ נוצר, מוקצית עבורו רשומת Inode, וה- UID של המשתמש שיצר את הקובץ נשמר ברשומת Inode זו.

הערה:
ה- User Name וה- UID מוקצים למשתמש על ידי ה- system admin.

© יצחק נודלר

או יותר מדויק: על ידי התכנית בה הוא 'משתמש' כדי ליצור/להגדיר חשבונות [= accounts] חדשים. היינו, ליצור/להגדיר משתמשים חדשים.
זהות זו שמוקצית למשתמש נשמרת בקובץ /etc/passwd.

GID (Group ID או: Group Name):

- בנוסף לכך שכל משתמש מזוהה על ידי User name ועל ידי UID, הרי שבלינוקס כל משתמש גם משויך לקבוצה ראשית.
- הוא יכול להיות בנוסף גם חבר בקבוצות משניות.
השיוך הזה נעשה על ידי ה- system admin, או יותר מדויק: על ידי התכנית בה הוא 'משתמש' כדי ליצור/להגדיר חשבונות [= accounts] חדשים.
- השיוך לקבוצות נשמר בקובץ /etc/passwd.

- כאשר קובץ נוצר, מוקצית עבורו כאמור רשומת Inode, וה- GID הראשוני של הבעלים של הקובץ נשמר בשדה ה- GID ב- Inode.

המשתמש root:

- בלינוקס ה- User Name של מנהל המערכת [= ה- system administrator] הוא "**root**".
- ה- UID שלו הוא 0.

- זהו משתמש מיוחס (Super User).
- מה פירוש 'מיוחס'? ה-Kernel מכיר אותו כמשתמש שיכול לעקוף את כל מגבלות ההרשאות.
- זאת אומרת, ה- Kernel מזהה את UID 0 כמי שיכול לדרוס כללי בעלות והרשאות, ומאפשר לו לשנות בעלות (באמצעות chown) או להריץ תוכניות מוגנות.
- דבר זה מקנה למעשה למנהל המערכת 'שליטה' מוחלטת על מערכת הקבצים.
- אחת הסיבות לכך שמנהל המערכת ישעה שימוש בפקודה {= ה- system program}: chown היא כדי לנסות שקם או לתקן קבצים פגומים.

חשיבות הבעלות: רגולציה ואכיפת חוק

שליטה על ה-Metadata (היכולת לשנות הרשאות):
הבעלים (או ה-root) הוא היחיד שיכול לשנות את שדה ה-Mode של קובץ. זה השמור ברשומת ה-Inode של הקובץ.
כלומר, לשנות את ההרשאות (באמצעות הפוקדה {= ה- system program}: chmod).

זה מונע ממשתמשים זרים להעניק לעצמם הרשאת כתיבה לקובץ של משתמש אחר.
זהו מנגנון הרגולציה של מערכת הקבצים.

© יצחק נודלר

אבטחה ברמת ה-Process:

תהליך חדש Child נוצר תמיד, חוץ מהמקרה של התהליך הראשון, על ידי כך שתהליך קיים, Parent, מבצע קריאת מערכת fork().

התהליך Child יורש [= inherits] את ה-UID של הבעלים [= ה- owner] של התהליך Parent.

כאשר התהליך Child מנסה לבצע קריאת מערכת (Syscall) לצורך ביצוע פעולה על קובץ F, כמו write() או read(), אזי ה- Kernel:

1. מאחזר את ה-UID של התהליך Child.
הוא מאחזר מידע זה מרשומת ה- PCB [= Process Control Block] של Child.
2. מאחזר את ה-UID של הקובץ F אליו Child מבקש לגשת.
הוא מאחזר מידע זה מרשומת ה- Inode של F.
3. משווה ובודק את ההרשאות.

מהתיאור הזה ניתן להבין כי הבעלות הופכת את האבטחה לפונקציה של ה-Kernel !!
<u>ולא</u> לפונקציה של תוכנת המשתמש.

© יצחק נודלר